

Transformer Pre-Filter for GDPR PII Detection

Results summary — Max Seidlitz · Deep Learning and Decision Making (TUM), Bosch GDPR case study · Team meeting 25.08.2026

This document summarises the work on the transformer pre-filter work package: what was built, the results on two datasets, and the open finding that needs a team decision. All numbers are pulled directly from the run artefacts on branch `claude/bert-prefilter-gdpr-pii-q35v0w`.

What the pre-filter does

It sits between Sweep 1 (Presidio + spaCy + regex) and the LLM review stage, and decides which of Sweep 1's *ambiguous* documents actually need an LLM call. A DistilBERT encoder scores each document; a three-zone router then auto-decides the confident cases and escalates only the uncertain band to the LLM.

Success criterion: not accuracy. It is how far the LLM call volume drops while document-level recall stays at or above the rule-based Sweep 1 baseline — originally stated in the project brief as **0.9833**, and now also measured directly (see “The baseline, measured”). False negatives are more expensive than false positives under GDPR, so the router escalates whenever it is unsure.

Headline result

The one comparison to remember from this deck

Measured on the same 210 test documents, with the same split: the pre-filter keeps **27 of 28** positive documents (recall 0.9643) while routing **0%** to the LLM. Sweep 1 keeps 26 of 28 (recall 0.9286) while routing **76.7%**. Better recall and no LLM spend, against a baseline that costs three quarters of the corpus in API calls.

The 0.9833 baseline quoted in the project brief could not be reproduced on this dataset — Sweep 1's own recall here is 0.9286. See “The baseline, measured” below.

Delivered

- Full pre-filter module: data loading & split repair, dual-head DistilBERT model, three-zone threshold calibration, training loop, inference/output writer, error-slicing report, EDA, run comparison figures — all committed and pushed.
- Verified end-to-end through the team's own evaluate pipeline twice, on both datasets.
- 51 automated tests: 21 on the routing logic (synthetic cases with a known-correct answer), 30 on documentation consistency.
- Seven findings for the team about the existing pipeline (three confirmed from the original brief, four new), documented in `classification/prefilter/README.md`.
- Sweep 1 run end-to-end on the same dataset for the first time, giving a baseline that is actually comparable rather than quoted from the brief.

Two training runs

The pre-filter was trained twice, on two very different datasets. The comparison is the point: it shows what changes when the dataset changes, not just a single number.

	Run 1 — pilot	Run 2 — scaled dataset
Dataset	500 rows (repo)	1,400 rows (Sonja's branch)
Language	English only	English + German (50/50)
Model	distilbert-base-uncased	distilbert-base-multilingual-cased
Parameters	66,963,469	135,334,669
max_length	256 tokens	512 tokens
Split used	stratified fallback (repo split unusable)	Sonja's own recommended_split
Epochs / selected	8 / epoch 2	5 / epoch 5
Training time	824s (~14 min)	4603s (~77 min)

Why Run 1 alone is not a usable result

The 500-row pilot is close to linearly separable: negative documents score 0.00–0.05, positive documents score 0.93–1.00, and the entire band between them is empty. There is no uncertain zone left to route, so the model reaches perfect validation and test metrics (accuracy/precision/recall/F1 all 1.0000) and routes **0.0%** of documents to the LLM at every recall target from 0.80 to 1.00. That is a real result for this dataset, but it says the synthetic pilot separates positives lexically (literal names, emails, IBANs vs. placeholders like “Cost Center Aggregate”) — not that the pipeline works. Run 1 mainly served to prove the code is correct before trusting it on real numbers.

Run 2: results on the 1,400-row dataset

This is the first run calibrated on positives spread across all three splits (Sonja's dataset fixed the pilot's all-negative validation/test problem), so it is the first run whose numbers are directly comparable to the rest of the team's pipeline.

Metric	Validation	Test
Accuracy	0.9952	0.9905
Precision	1.0000	0.9643
Recall	0.9643	0.9643
F1	0.9818	0.9643
PR-AUC	1.0000	—
Documents routed to LLM	0.0%	0.0%
Pre-filter recall (safety metric)	1.0000	0.9643
Missed positives	0	1

The project brief stated the rule-based baseline as 0.9833; that figure could not be reproduced on this dataset and should not be compared against directly. The measured Sweep 1 baseline on this exact split is in "The baseline, measured" below — the pre-filter is above it.

What happened

The router calibrated a threshold of $t_{\text{low}} = t_{\text{high}} = 0.01$ on the validation split (28 positive documents), where it achieved recall = 1.0 while routing nothing to the LLM. Applied to the held-out test split (28 different positive documents), that same threshold missed one:

- SYN-GENERAL_DOCUMENT-0018 — a document whose only PII is a DATE_TIME entity, scored $p = 0.0015$ by the model (essentially "no PII"), silently classified as non-PII with no LLM review.
- One false positive on the same test split (SYN-MEETING_NOTES-0076, $p = 0.028$) — harmless for recall, costs one unnecessary local rejection.

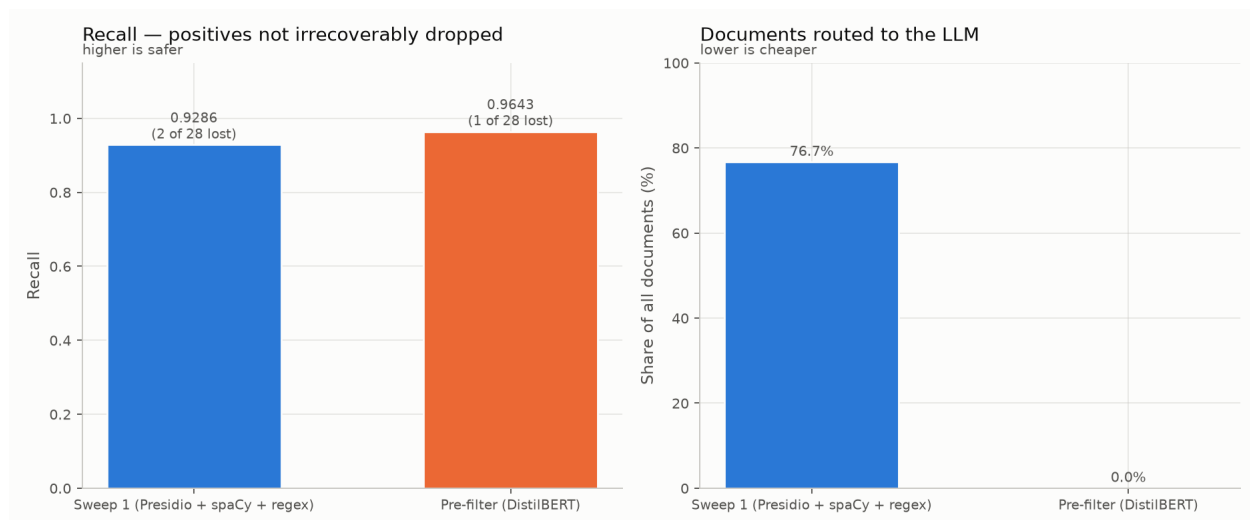
Root cause: the calibration is fit on only 28 positive validation examples. That is few enough that the recall-1.0 threshold sits right at the edge of what the validation sample can support, and does not generalise to the test split. This is a sample-size problem in the calibration step, not a bug in the routing logic (21 unit tests on the router's arithmetic all pass) and not a training problem (validation F1 was already 0.982 by the final epoch).

The baseline, measured

The 0.9833 recall this work package is measured against came from the project brief, not from a run anyone could point at — unknown dataset, unknown Sweep 1 configuration. Sweep 1 has now been run end-to-end (regex + Presidio + spaCy, en_core_web_lg + de_core_news_sm) over the same 1,400-row dataset, writing into the same run directory, so the evaluation scores every strategy on identical documents and an identical split.

Strategy	Accuracy	Precision	Recall*	F1	LLM calls
bert_prefilter (DistilBERT)	0.9952	1.0000	0.9643	0.9818	0.0%
rule_plus_bert (routed)	0.9905	0.9643	0.9643	0.9643	0.0%
rule_based (Sweep 1)	0.9095	1.0000	0.9286	0.4865	76.7%

**Recall here is the recoverable definition for every row: the share of positive documents the stage does not irrecoverably drop. A document Sweep 1 escalates to the LLM is not lost, so scoring it by strict detected_pii (0.3214) understates it badly and would flatter the pre-filter dishonestly. Accuracy, precision and F1 are the evaluation's own figures at the standardised prediction column.*



Same 210 test documents, same split. Left: recall, measured the recoverable way for both. Right: share of all documents sent to the LLM. The pre-filter is better on both axes at once — it is not trading recall for cost.

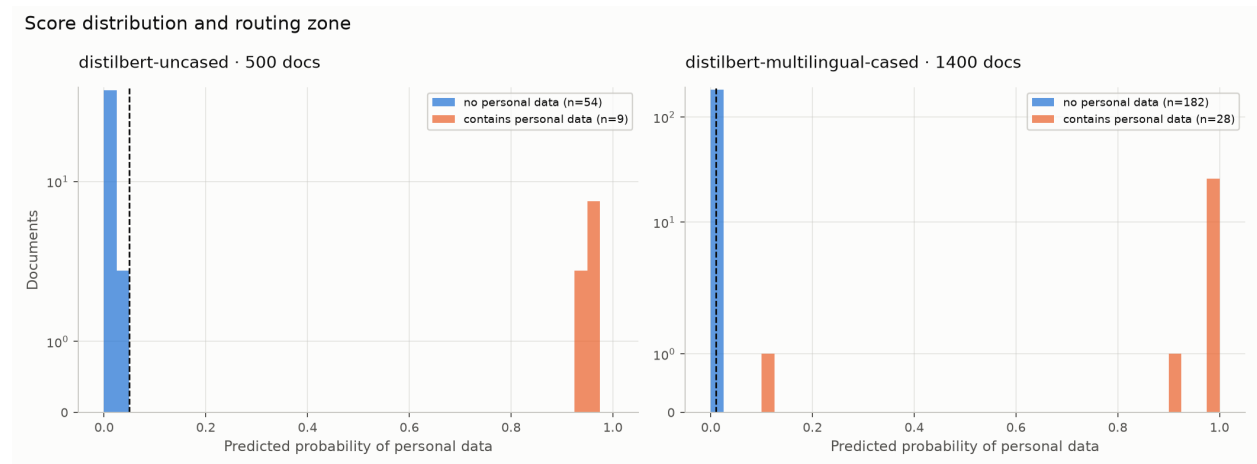
What this changes

- **The pre-filter beats Sweep 1 on both dimensions simultaneously.** Higher recall (0.9643 vs. 0.9286) and 76.7% fewer LLM calls. That is the case for keeping it.
- **The 0.9833 figure is not reproducible on this data.** Sweep 1's recall here is 0.9286 recoverable, or 0.3214 by strict detected_pii. Whatever the brief measured, it was not this dataset or not this metric — so “the pre-filter is below baseline” was comparing two numbers that were never commensurable.
- **The calibration finding still stands.** That the threshold does not generalise from 28 validation positives to the test split was measured validation-to-test *within* one run, and is untouched by any of this.
- Sweep 1 loses 2 of 28 positives outright; the pre-filter loses 1. Sweep 1's 19 undetected positives are mostly escalated rather than lost, which is exactly why the strict figure misleads.

Runtime: Sweep 1 is 32.8 ms/document, the pre-filter 133.1 ms/document, both on CPU. The pre-filter is four times slower locally and removes an LLM round-trip for three quarters of the corpus.

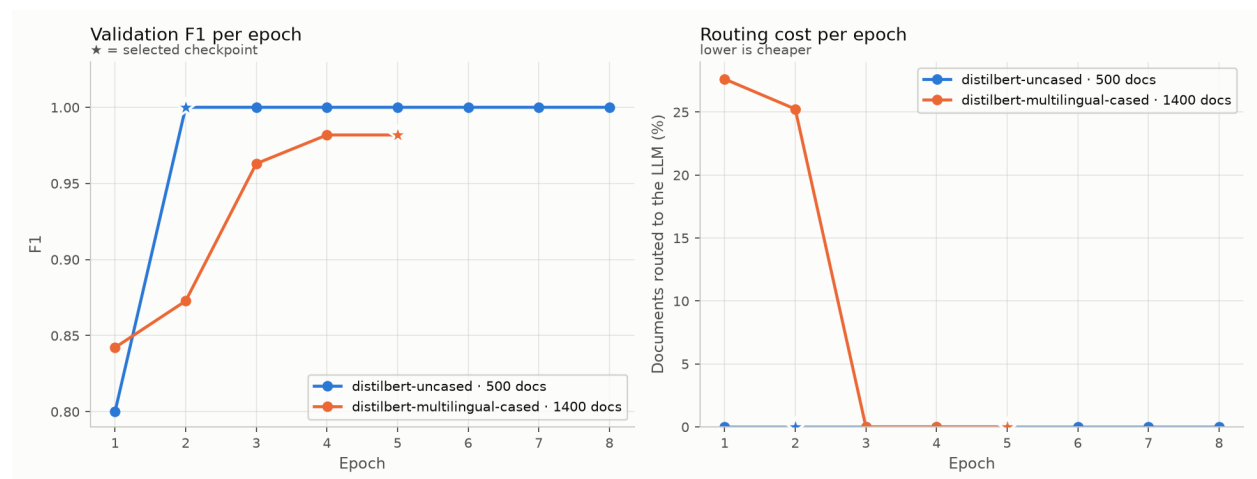
Figures

Score distribution — why the routing rate is 0% on both runs



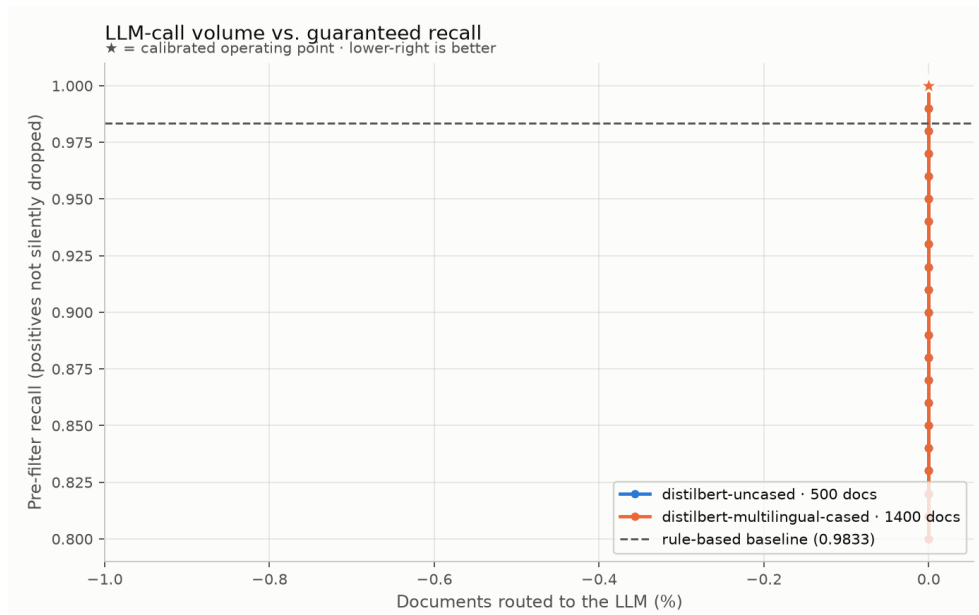
Predicted probability by true class, validation split. Blue = no personal data, orange = contains personal data. On the pilot (left) the classes are fully disjoint (gap 0.05–0.93). On the 1,400-row set (right) they are still close to disjoint, with one positive at $p \approx 0.11$ — the near-miss that foreshadows the test-split failure.

Training curves



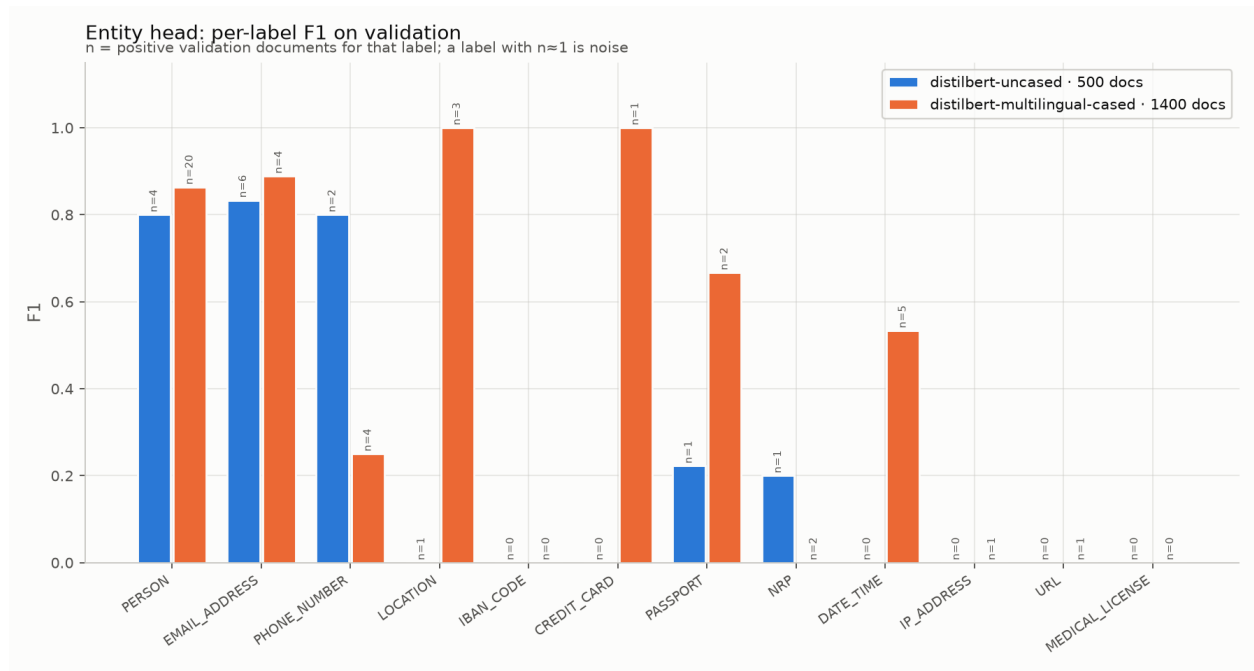
Left: validation F1 per epoch. Right: share of documents routed to the LLM per epoch. Run 2 (orange) needs more epochs to reach the same F1 as Run 1 (blue) — the harder, more realistic dataset. Both runs converge to 0% routing once the classes separate.

Routing frontier



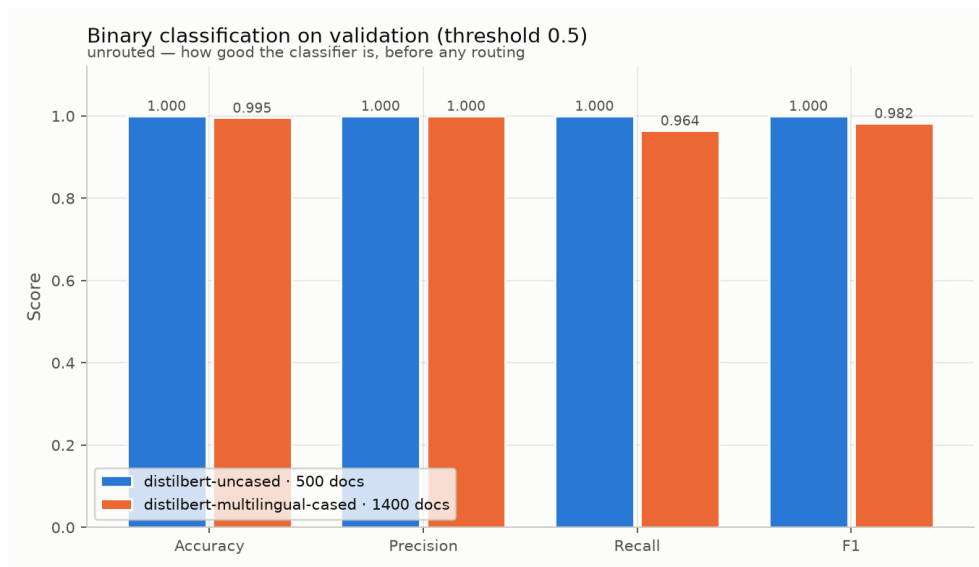
LLM-call share vs. guaranteed recall, swept across recall targets 0.80–1.00, on validation. Both runs are flat at 0% — on validation, neither dataset currently forces any routing. The dashed line is the 0.9833 baseline. This is exactly the plot that becomes meaningful once the validation sample is large enough to contain real uncertainty.

Entity head: per-label F1 (validation)



12-label multi-label head. *n* = positive validation documents for that label — a bar with *n*=1 is noise, not a measurement. More data helped PERSON (0.80→0.86 on 20 vs. 4 examples), unlocked LOCATION and CREDIT_CARD (1.0, but *n*=3 and *n*=1), and made DATE_TIME measurable for the first time (0.53). IBAN_CODE and MEDICAL_LICENSE still have zero validation positives in both runs. PHONE_NUMBER regressed (0.80→0.25).

Binary classification headline



Accuracy / precision / recall / F1 at the plain 0.5 threshold, validation split, both runs.

Findings for the team

Verified against the repo before training began. The first three are the points from the original brief; the rest came out of inspecting the data.

#	Finding	Status	Concerns
1	classification.config1 does not exist — evaluate is broken on main	Confirmed	Aissata
2	EVALUATION_SPLITS says 'eval', dataset says 'validation' (currently dead code)	Confirmed	Aissata
3	Repo dataset is the old 500-row pilot, not the 1,400-row set	Confirmed	Info
4	recommended_split had 0 positives in val/test on the pilot — fixed in Sonja's 1,400 set	New, resolved	Sonja
5	171/500 duplicate documents span split boundaries in the pilot	New	Sonja
6	difficulty==medium and challenge_category==medical_context are 100% positive (label leak in metadata)	New	Info
7	cost_analysis.py reads keys runtime.py never writes — cost columns are always zero	New, minor	Aissata

Full detail, code references and rationale for each: `classification/prefilter/README.md`, section “Findings for the team”.

Also investigated: Sonja's data-generation pipeline

Checked whether we could generate more training data ourselves, specifically for the under-represented entity types (IBAN_CODE, CREDIT_CARD, MEDICAL_LICENSE all have 0 validation positives). Two blockers:

- Production generation needs an LLM call (OpenRouter) for realistic text. This environment has neither an API key nor network access to openrouter.ai — blocked by the execution sandbox, same as huggingface.co earlier.
- Even with access, the generator has no weighting mechanism: entity-type combinations are hard-coded per document scenario and allocated round-robin. Getting more of a specific rare label requires changing Sonja's SCENARIO_ENTITY_COMBINATIONS config, not a flag we can pass.

Also found: `difficulty`, `edge_case` and `challenge_category` are never set by the current generator — every one of the 1,400 rows is easy / none. Explains why the new dataset has no graded difficulty, unlike the old pilot.

For discussion today

The decision this deck needs from the team

The pre-filter beats the measured Sweep 1 baseline on both recall and LLM-call volume, so the case for integrating it stands. What remains open: its threshold was calibrated on only 28 validation positives and missed 1 of 28 on the held-out test split (recall 0.9643, not below any reproducible baseline, but not 1.0 either). The test split has already been examined once (that is how the miss was found), so re-tuning the threshold now to catch that one document would be fitting to the test set — not a legitimate fix. Recommendation: report this honestly, and treat it as the argument for prioritising more (and better-balanced) validation data over further modelling for now.

Options, roughly in order of soundness

- **More data (real fix).** The calibration is unreliable specifically because validation has only 28 positive documents. Scaling the dataset further would widen that sample directly.
- **Statistically principled margin, applied blind.** Choose a lower-confidence-bound threshold from validation statistics alone (before looking at any new test data), then verify on a fresh, never-before-seen split. Cannot be applied to the current test split — it is already burned for this purpose.
- **Cross-validation across folds** to see how much the calibrated threshold actually moves between resamples of the same 1,400 rows — would quantify how much to trust it at all.
- **Not recommended:** adjusting the threshold specifically to recover the one missed document now that we know which one it is. That is tuning on the test set.

Status of the code

- Full pipeline verified end-to-end via the team's own evaluate command, on both datasets.
- 51 automated tests passing (21 routing logic, 30 documentation consistency).
- Everything committed and pushed to `claude/bert-prefilter-gdpr-pii-q35v0w`, draft PR open.
- Inference cost: 63.9 ms/document (Run 1, 256 tokens) vs. 133.1 ms/document (Run 2, 512 tokens, larger multilingual model) — both on CPU.
- Sweep 1 now runnable end-to-end in this environment too (Presidio + spaCy models fetched and installed), so the baseline comparison on the previous page is reproducible, not quoted.

Full detail, all findings, output-interface contract and reproduction steps: `classification/prefilter/README.md` on the branch above.